

A Verifiable and Efficient Symmetric Searchable Encryption Scheme for Dynamic Dataset With Forward and Backward Privacy

Xiaojie Zhu¹, Senior Member, IEEE, Jiancong Zhou², Yueyue Dai³, Member, IEEE, Peisong Shen⁴, Shabnam Kasra Kermanshahi, and Jiankun Hu⁵, Senior Member, IEEE

Abstract—The adoption of symmetric searchable encryption (SSE) has become increasingly common. However, many current SSE schemes assume an honest-but-curious cloud service provider (CSP) or necessitate significant overhead to manage a malicious CSP. Furthermore, most of these schemes are tailored for static datasets. Our paper presents an efficient SSE scheme that aims to address these challenges. To the best of our knowledge, this is the first scheme that supports dynamic datasets with forward and backward privacy, integrity verification of non-empty and empty search results, efficient search, non-interactive, light client, and both forward and inverted indexes simultaneously. In this paper, we present two novel approaches, Hexie and Jianding. Hexie implements secret sharing to conceal index entries, enabling dynamic updates, non-interactive interactions, and lightweight clients. To enhance the reliability of search results and address the problem of empty, incomplete, or inaccurate outcomes, we introduce the Jianding scheme as an extension of Hexie. It combines a chained MAC structure with a secret sharing scheme, which enables a client to verify the data integrity of the search result efficiently. Moreover, we propose graph-based dictionary sharding to enhance search efficiency. Finally, we conduct comprehensive experiments to validate the effectiveness of the proposed schemes.

Index Terms—Symmetric searchable encryption, dynamic dataset, verification, efficiency, cloud.

I. INTRODUCTION

WITH the booming development of cloud computing, more and more applications are migrated to the cloud,

Received 26 September 2023; revised 16 September 2024; accepted 19 December 2024. Date of publication 23 December 2024; date of current version 15 May 2025. A preliminary short version of this paper was published at the 22nd International Conference on Cryptography in India (Indocrypt 2021) [DOI: 10.1007/978-3-030-92518-5_22]. (Corresponding author: Xiaojie Zhu.)

Xiaojie Zhu is with the King Abdullah University of Science and Technology, Thuwal 23955, Saudi Arabia (e-mail: xiaojie.zhu@kaust.edu.sa).

Jiancong Zhou is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100045, China, and also with School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: zhoujiancong@iie.ac.cn).

Yueyue Dai is with the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: yueyuedai@ieee.org).

Peisong Shen is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100045, China (e-mail: shenpeisong@iie.ac.cn).

Shabnam Kasra Kermanshahi and Jiankun Hu are with the School of Systems & Computing, University of New South Wales, Canberra, NSW 2600, Australia (e-mail: s.kasra_kermanshahi@unsw.edu.au; j.hu@unsw.edu.au).

Digital Object Identifier 10.1109/TDSC.2024.3521423

e.g., healthcare [2], location-based service [3], and Internet of Things [4]. Meanwhile, privacy issues emerge as crucial concerns. For example, the National Electoral Institute of Mexico experienced that 93 million voter registration records stored at the Amazon cloud were compromised [5].

To reduce privacy violation cases, various techniques have been proposed. Among them, a cryptographic notion named symmetric searchable encryption (SSE) is favored. SSE is different from traditional encryption, e.g., AES and DES. It offers not only data confidentiality but also ciphertext (encrypted data) searchability. By using SSE, users can encrypt their data before outsourcing them to the cloud. Moreover, the cloud service provider (CSP) can serve the user by directly searching over the ciphertext.

However, most existing SSE schemes [6], [7], [8], [9] are proposed under the assumption that the CSP is honest but curious. Based on the assumption, the CSP honestly and correctly executes the protocol. In practice, the assumption is not always held. The violation of the protocol may be subject to external attacks, internal misconfiguration errors, software bugs, and even insider threats [10], [11]. Exemplary consequences are CSP executing a fraction of search operations or omitting some files in the search result.

In order to efficiently address the above issues, a large number of investigations have been conducted to ensure data integrity against a malicious CSP [12], [13], [14], [15]. For SSE, the verifiability of SSE also has been raised and captures more and more attention [11], [16], [17], [18], [19]. Unfortunately, as shown in Table I, the existing approaches fail to address at least one of the following unique challenges in SSE.

Dynamic dataset: It is a common operation to update datasets, including adding and removing data. Dynamic symmetric searchable encryption (DSSE) conducts the insert and delete operations over encrypted data. As new data gets added to a dataset stored on the CSP, information gets exposed to the CSP, potentially revealing the timestamp of a new record's addition, the memory address of an inserted record, etc. Moreover, if the CSP records previous queries and observes states before and after insertion, it may be able to link the document to a specific keyword. To prevent the above leakage, forward privacy [20] has been proposed. It requires the inserted data unlinked to the previous searches. Exposed data during a delete operation

TABLE I
COMPARISON WITH EXISTING SSE SCHEMES

schemes	dynamism	integrity of search result	robust against malicious empty search result	forward index	SE	RT	QGE	UE	FP	BP
SSE [7]	×	×	×	×	$O(n_w)$	1	$O(1)$	×	×	×
$\Sigma\sigma\phi\sigma\varsigma$ [33]	✓	×	×	×	$O(n_w)$	1	$O(1)$	$O(1)$	✓	×
FIDES [10]	✓	×	×	×	$O(a_w)$	1*	$O(1)$	$O(1)$	✓	II
MITRA [30]	✓	×	×	×	$O(a_w)$	1*	$O(a_w)$	$O(1)$	✓	II
ORION [30]	✓	×	×	×	$O(n_w \log^2 N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	✓	I
AURA [28]	✓	×	×	×	$O(n_w)$	1	$O(\log D)$	$O(1)$	✓	II
VSPS [34]	✓	✓	×	×	$O(m \log^3 N)$	1	$O(1)$	$O(\log^2 N)$	✓	×
VFSSE [36]	✓	✓	×	×	$O(a_w)$	1	$O(1)$	$O(1)$	✓	×
AATVSSE [37]	✓	✓	×	×	$O(D)$	1*	$O(1)$	$O(W)$	✓	II
GSSE [35]	✓	✓	✓	×	$O(\log W)$	1	$O(1)$	$O(1)$	×	×
FTVSSE [32]	✓	✓	✓	×	$O(a_w)$	1	$O(1)$	$O(1)$	✓	×
FPVSSE [31]	✓	✓	✓	✓	$O(a_w + n_w)$	1	$O(1)$	$O(1)$	✓	×
Hexie Section 4.1	✓	×	✓	✓	$O(a_w)$	1*	$O(1)$	$O(1)$	✓	II
Jianding Section 4.2	✓	✓	✓	✓	$O(a_w)$	1*	$O(1)$	$O(1)$	✓	II
Distributed Jianding 5	✓	✓	✓	✓	$O(a_w/ S)$	1*	$O(1)$	$O(1)$	✓	II

SE: search efficiency. RT: round trip. UE: update efficiency. QGE: query generation efficiency. FP: forward privacy. BP: backward privacy

N , D , and W denote the total number of keyword/document pairs, the total number of documents, and the total number of distinct keywords in the dataset. a_w and n_w denote the total number of updates on the keyword w and total number of matching documents. S represents the number of parallel threads. * denotes the number of round trips required to retrieve matching document identifiers in the result-hiding scheme.

could potentially reveal deletion time and the memory address of a deleted record. Moreover, a deleted record could easily be linked to a searched keyword by analyzing the search process before and after the deletion operation. In order to mitigate these attacks, backward privacy [21] is proposed. It demands the deleted document undisclosed by the search operations before and after the delete operation.

Integrity of search result: The search result returned from the CSP may be incorrect due to deliberate alteration or unpredicted data loss. For example, instead of returning target documents, CSP may send the client advertisement. Moreover, the hardware or software is not always stable. It may result in data loss due to the failure of hardware or software. Additionally, it is common to lose packets during communication.

Malicious empty search result: The malicious CSP may deliberately return the empty result for self-interest, e.g., saving bandwidth and energy. However, the existing verifiable SSE schemes, based on common verification techniques, such as Merkle hash tree [22], message authentication code [23] and RSA accumulator [24], can not resist such attack [21], [25], [26].

Efficient search: Entering the era of Big Data, the volume of the data increases dramatically. It is desirable to design an SSE scheme where search time is independent of the size of the whole dataset. A promising approach is to set a secure index [25], [27] while limiting the information leakage to CSP. However, many existing schemes [6], [7], [28] still cannot obtain such property, which sequentially searches over all the documents.

Light client: The goal of outsourcing the dataset to CSP is to reduce the workload of the client. However, many SSE schemes require a heavy client to support query generation or search result verification. For example, Sun et al. proposed schemes to achieve forward and backward privacy by increasing the cost of query generation heavily [28], [29]. Similarly, the time complexity of query generation for a keyword in MITRA [30] and FIDES [10] is linear to the number of updates. In counter-based dynamically verifiable searchable encryption schemes [31] [32], the client needs to maintain additional counters to record the number of keyword updates for verifying search results. It is

desirable to have query generation with a computational complexity of $O(1)$ and no extra counters required for search result verification.

Non-Interactive: It is desirable to have non-interactive property, which not only reduces bandwidth cost but also paves the way for high throughput. Moreover, it enables a client to be offline once the request has been sent to the CSP. However, most existing schemes achieving forward and backward privacy need many rounds of communication between client and CSP. For example, Oblivious RAM (ORAM) offers an opportunity to achieve forward and backward privacy while requiring many rounds of communication between the client and CSP [10], [30]. It is important to clarify that in the paper, “non-interactive” specifically refers to the search process over indexes, in line with the definitions in [33], [34]. This approach facilitates a fair comparison between result-hiding SSE and result-revealing SSE, as both share the same search procedure.

Availability of both forward and inverted index: The inverted index is designed for keyword search while the forward index is built for document search. The combination of forward and inverted indexes offers both search flexibility and efficiency. Additionally, this index structure significantly reduces search time. To fully leverage these advantages, this structure could be applied in scenarios such as libraries, where the data set is large and stable, with infrequent updates and a high volume of retrieval requests. The existing dynamic SSE schemes [10], [28], [30], [33], [35] only consider the requirement of keyword search while ignoring the importance of forward index in adding/deleting a document. Guo et al. recognized the significance of the forward index and made an attempt to construct it [31]. However, their scheme does not support searching through file identifiers. For the inverted index, to delete a document, it requires to retrieve all the entries of the inverted index, e.g., $(w_1, id_1), \dots, (w_n, d_1)$. However, in the forward index, it only requires to retrieve one entry of the forward index, e.g., $(id, w_1, \dots, w_n)$.

In this paper, we propose the first symmetric searchable encryption to support all the above-mentioned aspects simultaneously. Our paper’s contributions are summarized below:

- 1) To the best of our knowledge, we propose the first DSSE scheme (Hexie) capable of simultaneously supporting forward/backward privacy, integrity verification of non-empty and empty search results, efficient search, non-interactive to retrieve encrypted document identifiers, the light client with efficient query generation and low client storage, and both forward and backward indexes.
- 2) To ensure resilience against empty, incomplete, or inaccurate search outcomes, we proposed an extension to Hexie named Jianding. This involves merging a chained MAC structure with a secret sharing system to enable clients to verify the integrity of the search results.
- 3) The client only needs to store one secret share, which is comparable to the best counter-based dynamic SSE schemes [30], [38] and requires much less computation.
- 4) To enhance the search efficiency, we propose a graph-based dictionary sharding mechanism, considering constraints of the number of servers, server capacity, workload balance, and response time.
- 5) We formally analyze the security of the proposed schemes and demonstrate that our proposed schemes can attain our design goals. Besides, we conducted comprehensive experiments to evaluate the proposed schemes and dictionary sharding solution. The result shows the effectiveness of both proposed approaches. Especially, we compare our scheme with the most typical forward-secure scheme, $\sum o \varphi os$ [33], achieving more than 3000 and 10 times faster in search and update respectively.

The rest of this paper is organized as follows. In Section II, we introduce the related work. Then we review the preliminaries in Section III. After that, we introduce the proposed SSE schemes Hexie and Jianding in Section IV. In Section V, we introduce the dictionary sharding solution. We present the experimental results and security analysis in Sections VI and VII, respectively. Finally, draw the conclusion in Section VIII.

II. RELATED WORK

SSE has been the focus of active research in the recent twenty years. It develops from static SSE to dynamic SSE, and from dynamic SSE to verifiable SSE. In this section, we present the related work by following this line.

Static SSE: SSE was first proposed by Song et al. [6] in 2000. In their work, each keyword is encrypted with two layers. The first layer is implemented by pseudorandom permutation and used to hide the keyword. The second layer is realized by the pseudorandom function and applied to support encrypted keyword search. Four years later, Goh et al. [27] proposed a secure index to improve search efficiency. After that, Curtmola et al. [7] formalized the security definition of SSE and proposed two constructions corresponding to non-adaptive semantic security and adaptive semantic security. With the late development of static SSE, the types of query are enriched significantly, such as ranked keyword search [25], [39], fuzzy keyword search [40], similarity search [41], and semantic search [42].

Dynamic SSE: To support the dynamic dataset update, various dynamic SSE schemes were proposed [8], [43], [21], [44], [45],

[46], [47]. Kamara et al. [8] extended the inverted index to achieve not only dynamic dataset update, but also sub-linear search time and security against adaptive chosen keyword attack. One year later, they applied a red-black tree data structure to achieve a parallel keyword search and dynamic file insert and delete [43]. Cash et al. proposed T-set structure to store keyword/document identifier tuple [9]. To support the large-scale database update, they add a new database to store newly inserted documents and a revocation list to record the removed documents [44]. Stefanov et al. [21] proposed a hierarchical structure to store the keyword/document tuples, which achieves sub-linear search time and forward privacy. To support multi-keyword search, Xia et al. [46] proposed an tree based index structure. To sort the search result, Guo et al. [47] improved the inverted index structure by embedding the TF-IDF model.

Verifiable SSE: To prevent the cloud server from running incorrect search result, Chai et al. [16] proposed the first verifiable SSE scheme, where the search result includes not only retrieved documents but also proof of search process. To achieve stronger security, Kurosawa et al. [17] introduced the definition of universally composable (UC) secure SSE and presented an verifiable SSE to achieve the UC security. To support verifiability of dynamic SSE, Bost et al. [34] proposed a generic verifiable dynamic SSE scheme and combined it with the SSE scheme proposed by Stefanov et al. [21]. To improve efficiency, Ge et al. [37] designed accumulative authentication tag and embedded it into the dynamic SSE. Zhu et al. utilized Merkle Patricia Tree to extend the verifiability from single user to multiple users [35]. Jiang et al. [18] proposed Qset data structure to support multi-keyword verifiable search. Sun et al. [11] utilized bilinear mapping to realize the conjunctive keyword verifiable search. Guo et al. [31] applied puncturable PRF to achieve conjunctive keyword verifiable search. Zhu et al. [19] applied the RSA accumulator to implement fuzzy keyword verifiable search. Yuan et al. [32] utilized authenticated encryption to securely record the entire update history, extending verifiable keyword search to ensure fault tolerance against incorrect updates.

III. BACKGROUND

In this section, we introduce the concept of the inverted index, forward index, and secret sharing.

A. Inverted Index

According to the inverted index definition from the National Institute of Standards and Technology (NIST), an inverted index is an index into a set of texts and each index entry gives the word and a list of texts, possibly with locations within the text, where the word occurs [48].

In the paper, a set of texts consists of documents and formally, we define the inverted index in Definition 1, where (w, L) is the entry of the inverted index, w denotes a keyword, L represents a list of document identities, and $D[id]$ means the words contained in the document having document identity id .

Definition 1 (Inverted index): Let DB be a set of documents. An index I of DB is an inverted index if and only if $\forall (w, L) \in I$ matches $id \in L$ and $w \in D[id]$.

B. Forward Index

The forward index is also an index of documents. In contrast to the inverted index, each entry maps the document's identity to words. Formally, we define the forward index in definition 2, where W is a set of words.

Definition 2 (Forward index): Let DB be a set of documents. An index I of DB is a forward index if and only if $\forall(id, W) \in I$ matches $id \in DB$ and $W \in D[id]$.

C. Secret Sharing

Secret sharing is a method for distributing a secret among a group, in such a way that no individual holds any intelligible information about the secret, but when a sufficient number of individuals combine their shares, the secret can be reconstructed [49]. According to [50], the t -out-of- n secret sharing scheme is formalized in definition 3, where t is the minimal number of individuals required to recover the secret and n is the total number of participants.

Definition 3 (t -out-of- n secret sharing scheme): A t -out-of- n secret sharing scheme over message space M is a pair of algorithms (Share, Reconstruct) such that:

Share is a randomized algorithm that on any input $m \in M$ outputs a n -tuple of shares $(s_1, \dots, s_n)$.

Reconstruct is a deterministic algorithm that given a t -tuple of shares outputs a message in M , and satisfying the following correctness requirement:

$$Pr_{Share(m) \rightarrow (s_1, \dots, s_n)}[Reconstruct(s_{i_1}, \dots, s_{i_t}) = m] = 1 \text{ for } \forall m \in M, \forall S = \{i_1, \dots, i_t\} \subseteq \{1, \dots, n\} \text{ of size } t.$$

From above construction, we can observe that the randomness inherent in the shares is required by the encrypted keywords, mirroring their confidential nature. Furthermore, the steadfastness of the secret aligns ideally with the necessity for dependable search outcomes.

In the paper, we set t equal to n . It means all the shares are required to recover the secret. Taking the inverted index an example, we assign a secret to each keyword and each document containing the keyword holds a share. The document retrieval process can be considered a secret recovering process. Detail is presented in Section IV.

IV. PROPOSED SSE SCHEMES

In this section, we first propose a dynamic SSE scheme with forward and backward privacy, named Hexie. To further enhance the scheme to be robust against the modification of the search process and results, we then propose Jianding.

A. Hexie: A Dynamic SSE Scheme With Forward and Backward Security

In this section, we first present the overview of Hexie and then demonstrate each component of its construction.

1) **Notations:** Before giving the description of the proposed schemes, we show the frequently used notations on Table II.

2) **Overview of Hexie:** For an inverted index, we assign each keyword a secret, and the secret is further split into many secret shares. For each secret, the number of secret shares is equal

TABLE II
NOTATIONS

λ	the security parameter
DB	a database
id	a document identifier
ids	a list of document identifiers
w	a keyword
W_{id}	the keywords included in document id
EI_{idx}	the list stores encrypted inverted index
EF_{idx}	the list stores encrypted forward index
k	a secret key
DI	a list stores the last secret share for each keyword
DF	a list stores the last secret share for each document identifier

to the total number of correlated documents plus one. Each document is assigned a secret share and the secret share is used to encrypt the document identifier. The last secret share is kept to support future operations, either insert or delete. Since the last share can be considered as a new secret, it also can be split into many shares as before. Similarly, we also conduct analogous operations for the forward index. The difference is that we assign a secret to a document identifier instead of a keyword and each keyword contained in the document is encrypted by a secret share. The search operation is a process to recover the secret. In the inverted index, with the input of a keyword, all the corresponding document identifiers are retrieved. Meanwhile, the corresponding secret shares are recovered, which merge with the last secret share to recover the secret. For the forward index, the search process is the same except for the input. To support efficient search and light client, we set all the secrets to be identical, either for the inverted index or forward index. If the applied secrets are different, all the secrets have to be stored so that they can be used as a stop signal for search and correctness proof. In our construction, we set the secret to 1^λ , where λ is a security parameter.

3) **Setup of Hexie:** Let $DB = \{(id_i, W_i) : 1 \leq i \leq D\}$ be a database with $id_i \in \{0, 1\}^\ell$, $W_i \subseteq \{0, 1\}^*$. Here, id_i are document indices and W_i is a set of keywords matching document id_i . We denote the set of keywords in DB with $W = \cup_{i=1}^D W_i$. We define $N = \sum_{i=1}^D |W_i|$ as the total number of document/keyword pairs. We denote $DB(w) = \{id_i | w \in W_i\}$ as the set of documents containing keyword w . The setup algorithm is the initial procedure of Hexie. As shown in Algorithm 1, four different lists are initialized. Two lists DI and DF are built for storing the helpful data for searching encrypted inverted index and forward index respectively. In respective of encrypted index, EI_{inx} is built for storing encrypted inverted index while EF_{inx} is for forward index. Additionally, a secret key k is randomly chosen with the size of λ bits.

Note that our scheme supports two different operation types, insert and delete. For the insert operation, the value of the operation type is set to 0 while for the delete operation, the value of the operation type is set to 1. The adversary is incapable to distinguish them from each other since the value is encrypted by a semantically secure encryption algorithm, detailed in Section IV-A4.

4) *Update of Hexie*: The update of *Hexie* consists of the encrypted inverted index EI_{inx} update and the encrypted forward index EF_{inx} update. For each update operation, it could be either insert or delete operation. For example, consider a document with the identifier 1 that contains the text “hello world”. In this case, the inverted index comprises two entries: (“hello”, 1) and (“world”, 1). The forward index contains a single entry: (1, (“hello”, “world”)). After the keyword “hello” is removed from the document, the inverted index is updated to only include one entry: (“world”, 1), while the forward index is modified to (1, (“world”)). In this section, we present the details of the update process.

As shown in Algorithm 2, encrypted inverted index update requires the involvement of both the client and server. On the client side, we first extract a pair of a keyword w and corresponding document identifier lists ids from database DB . Given the keyword w , the pseudorandom function F is invoked and the output is equally split into two parts as two keys, k_1 and k_2 . For each document identifier id in the list ids , we first get the value π_1 from DI with the key w . If the value of π_1 is empty, the keyword is new and requires to be assigned a new secret. Otherwise, π_1 is the last secret share of the assigned secret. After that, k_2 is applied as a secret key to encrypt the concatenation of the operation type op and document identifier id by invoking the symmetric encryption algorithm Enc_2 . The output is denoted as c_{id} . After encryption, a random string r is generated and XORed with the π_1 . The result is rewritten to π_1 as a new secret share that will be stored back to DI where the corresponding key is w . To generate the key of the final output tuple, the PRP is called with the input k_1 and π_1 and the result is XORed with the random string r . The output c_a combines with the c_{id} as the value, corresponding to the key c_w . Finally, the client sends the tuple $(c_w, (c_a, c_{id}))$ to the server. Upon receiving the submitted pair, the server directly put the data to the EI_{inx} where the key is c_w and the corresponding value is c_a, c_{id} . The update of the encrypted forward index is similar to the above description. The detail is presented in Algorithm 3.

Note that in the above update algorithm the operation of adding/removing (keyword, document identifier) or (document identifier, keyword) cannot be linked to prior operations and does not reveal extra information. In dealing with the case to delete a complete document, to avoid extensive updates to both indexes, the client only needs to store the document identifier locally until executing the compaction operation (detailed in Section IV-A8). The cost of maintaining the list is determined by the number of complete document deletion operations performed before a compaction operation. For each such operation, the deletion list only needs to store the corresponding document identifier.

5) *Trapdoor Generation of Hexie*: To launch a query, a client needs to generate a trapdoor. In *Hexie*, as shown in Algorithm 4, the trapdoor consists of two parts. The first component is the key $\bar{k}_{t,1}$ generated by invoking F with the input of secret key k and input inp , either a keyword or a document identifier. The second component is the string π read from the stored key-value structure, either DI or DF .

6) *Search of Hexie*: Given the trapdoor from the client, the server applies the trapdoor to search over stored data. As shown

Algorithm 1: Setup of Hexie.

Input: (λ)

Output: ($DI, DF, EI_{inx}, EF_{inx}, k$)

- 1: initialize a list DI to store local data for inverted index
 - 2: initialize a list DF to store local data for forward index
 - 3: initialize a list EI_{inx} to store encrypted data of inverted index
 - 4: initialize a list EF_{inx} to store encrypted data of forwarded index
 - 5: generate a secret key $k \leftarrow \{0, 1\}^\lambda$
-

Algorithm 2: Encrypted Inverted Index Update of Hexie.

Input: (EI_{inx}, DI, k, DB)

Output: (DI, EI_{inx})

1: **Client:**

2: **for all** $(w, ids) \in DB$ **do**

3: $k_1 \parallel k_2 \leftarrow F(k, w)$

4: **for** $id \in ids$ **do**

5: $\pi_1 \leftarrow DI[w]$

6: **if** π_1 is empty **then**

7: $\pi_1 \leftarrow 1^\lambda$ ▷secret assignment

8: $c_{id} \leftarrow Enc_2(k_2, op \parallel id)$

9: $r \leftarrow \{0, 1\}^\lambda$ ▷secret share

10: $\pi_1 \leftarrow \pi_1 \oplus r$

11: $c_w \leftarrow H(\pi_1)$

12: $c_a \leftarrow r \oplus PRP(k_1, \pi_1)$

13: $DI[w] \leftarrow \pi_1$ ▷last secret share

14: send $(c_w, (c_a, c_{id}))$ to server

15: **Server:**

16: $EI_{inx}[c_w] \leftarrow (c_a, c_{id})$

in Algorithm 5, two components $\bar{k}_{t,1}$ and π are first extracted from the trapdoor. After that, the collision-resistant hash function H is invoked with the input of π and the output is denoted by $\bar{c}$. If $\bar{c}$ is the key of EI_{inx} , then the search process is conducted over the encrypted inverted index, otherwise encrypted forward index. For brevity, we take the search over the encrypted inverted index as an example. Looking over EI_{inx} with the key $\bar{c}$, the corresponding value c_a and c_{id} is retrieved. After that, the PRP is invoked with input $\bar{k}_{t,1}$ and π . The output of PRP is XORed with c_a , resulting in r . The new value of π is computed via the π XORed with r . Before checking whether π is equal to 1^λ , c_{id} is stored into the set C . If π is equal to 1^λ , the search process is finished and C is returned to the client. Otherwise, the above search process is executed repeatedly.

7) *Decryption of Hexie*: Upon receiving the ciphertext from the server, the client runs Algorithm 6 to decrypt the retrieved ciphertext. For all ciphertext in C , the Dec_2 is invoked sequentially. Particularly, if the operation type is *delete*, the corresponding value is removed from the set S , otherwise, the value is added into the set S . Since the client maintains a deletion list, the final output must exclude the document identifiers contained within that list. As the entire update history is properly maintained in

Algorithm 3: Encrypted Forward Index Update of Hexie.

Input: (EF_{inx} , DF , k , DB)
Output: (DF , EF_{inx})

- 1: **Client:**
- 2: **for all** (id, W_{id}) $\in DB$ **do**
- 3: $k_3 \parallel k_4 \leftarrow F(k, id)$
- 4: **for all** $w \in W_{id}$ **do**
- 5: $\pi_2 \leftarrow DF[id]$
- 6: **if** π_2 is empty **then**
- 7: $\pi_2 \leftarrow 1^\lambda$ $\triangleright$ secret assignment
- 8: $c_w \leftarrow Enc_2(k_4, op \parallel w)$
- 9: $r \leftarrow \{0, 1\}^\lambda$ $\triangleright$ secret share
- 10: $\pi_2 \leftarrow \pi_2 \oplus r$
- 11: $c_{id} \leftarrow H(\pi_2)$
- 12: $c_a \leftarrow r \oplus PRP(k_3, \pi_2)$
- 13: $DF[id] \leftarrow \pi_2$ $\triangleright$ last secret share
- 14: send (c_{id} , (c_a, c_w)) to server
- 15: **Server:**
- 16: $EF_{inx}[c_{id}] \leftarrow (c_a, c_w)$

Algorithm 4: TrapGen of Hexie.

Input: (k , DI , DF , $inp \in \{id, w\}$)
Output: (trp)

- 1: $\pi \leftarrow \phi$
- 2: $(\bar{k}_{t,1}, \bar{k}_{t,2}) \leftarrow F(k, inp)$
- 3: **if** $inp \in DI$ **then** $\pi \leftarrow DI[inp]$
- 4: **if** $inp \in DF$ **then** $\pi \leftarrow DF[inp]$
- 5: $trp \leftarrow (\bar{k}_{t,1}, \pi)$

the search result, the decrypted result remains accurate even in the presence of duplicate keyword deletions.

8) *Compaction of Hexie:* In the above update algorithm, the delete operation does not remove the deleted documents from the storage. In this section, we attempt to tackle this issue.

We name the process to remove the deleted documents from the storage as compaction since deletion can be considered to find a pair of insert and delete operations and remove both of them. To remove the deleted documents, it requires index reconstruction. During the index reconstruction, all the deleted documents are removed from the server and corresponding entries are removed from the index. In detail, the data owner first downloads and decrypts the encrypted index, and then properly removes the deleted entries. Finally, the refined index is re-encrypted and stored back to the server.

It's important to note that the decision to run compaction should be based on careful analysis and evaluation of the system's performance. Compaction is a computationally expensive operation, and its benefits should outweigh the potential impact on system performance. Therefore, it is crucial to strike a balance between the need for compaction and the overall system responsiveness.

9) *Correctness Analysis:* To prove the correctness of Hexie is to prove the secret sharing scheme is properly embedded into the searchable encryption. Particularly, the Setup and Update algorithms comply with the generation of secret shares while

Algorithm 5: Search of Hexie.

Input: (EI_{inx} , EF_{inx} , trp)
Output: (C)

- 1: initialize list C
- 2: $(\bar{k}_{t,1}, \pi) \leftarrow trp$
- 3: $\bar{c} \leftarrow H(\pi)$
- 4: **if** $\bar{c} \in EI_{inx}$ **then**
- 5: $c_a, c_{id} \leftarrow EI_{inx}[\bar{c}]$
- 6: $r \leftarrow c_a \oplus PRP(\bar{k}_{t,1}, \pi)$
- 7: $\pi \leftarrow \pi \oplus r$
- 8: $C \leftarrow C.add(c_{id})$
- 9: **while** $\pi \neq 1^\lambda$ **do**
- 10: $\bar{c} \leftarrow H(\pi)$
- 11: $c_a, c_{id} \leftarrow EI_{inx}[\bar{c}]$
- 12: $r \leftarrow c_a \oplus PRP(\bar{k}_{t,1}, \pi)$
- 13: $\pi \leftarrow \pi \oplus r$
- 14: $C \leftarrow C.add(c_{id})$
- 15: **if** $\bar{c} \in EF_{inx}$ **then**
- 16: $c_a, c_w \leftarrow EF_{inx}[\bar{c}]$
- 17: $r \leftarrow c_a \oplus PRP(\bar{k}_{t,1}, \pi)$
- 18: $\pi \leftarrow \pi \oplus r$
- 19: $C \leftarrow C.add(c_w)$
- 20: **while** $\pi \neq 1^\lambda$ **do**
- 21: $\bar{c} \leftarrow H(\pi)$
- 22: $c_a, c_w \leftarrow EF_{inx}[\bar{c}]$
- 23: $r \leftarrow c_a \oplus PRP(\bar{k}_{t,1}, \pi)$
- 24: $\pi \leftarrow \pi \oplus r$
- 25: $C \leftarrow C.add(c_w)$

Algorithm 6: Decryption of Hexie.

Input: (C , $k_d \in \{k_2, k_4\}$, $inp \in \{w, id\}$, DI , DF)
Output: (S)

- 1: initialize S
- 2: **if** $inp \in DI$ **then**
- 3: $\pi_1 \leftarrow DI[inp]$
- 4: **if** $inp \in DF$ **then**
- 5: $\pi_1 \leftarrow DF[inp]$
- 6: **if** $\pi_1 \neq \phi$ and $C = \phi$ **then**
- 7: **return** false $\triangleright$ robust against malicious empty result
- 8: **for all** $0 \leq i \leq |C|$ **do**
- 9: $op||s \leftarrow Dec_2(k_d, C_i)$
- 10: **if** $op=1$ **then** $\triangleright 1$ represents delete
- 11: $S \leftarrow S - s$
- 12: **else**
- 13: $S \leftarrow S \cup s$

the search algorithm complies with the secret recovery. In our scheme, the n-out-of-n secret sharing scheme is used. In the Setup and Update algorithms, the secret share r_i is randomly selected from $\{0, 1\}^\lambda$, and the last secret share r_n is set to $r_1 \oplus \dots \oplus r_{n-1} \oplus 1^\lambda$. In the search process, each secret share r_i is recovered and the secret 1^λ is finally reconstructed.

B. Jianding: A Verifiable Dynamic Searchable Encryption Scheme

The proposed Hexie supports both insert and delete operations. However, it is not capable to guarantee the completeness of the search result. For example, the cloud service provider may return partial search results to reduce energy cost. In addition, packet loss during transportation and hardware failure is unpredictable. In this section, we attempt to address the above issues by proposing a verifiable scheme extended from Hexie, named Jianding.

Hexie is suited for scenarios where verifying the completeness and correctness of retrieved results is not necessary, making it ideal for stable environments. In contrast, Jianding includes a search result verification mechanism that effectively addresses cases where searches might produce partial or incorrect results, making it suitable for unstable environments.

1) *Construction of Jianding*: The verifiability includes two aspects, search path verification and search result verification. Search path verification aims to guarantee that the server properly conducts the search operation. The search result verification is to guarantee the integrity of the retrieved ciphertext.

The construction of Jianding mainly differs from Hexie in adding the chained MAC structure. It involves the modification of Update, and Decryption algorithms while the Setup and Search algorithm is not affected. In the update, the difference between Jianding and Hexie is the chained MAC computation. Specifically, we append the MAC of the first ciphertext to the content of the second ciphertext. This procedure is executed iteratively until the last ciphertext is generated and the MAC of the last ciphertext is stored in the DI for the inverted index and the DF for the forward index. The update of Jianding consists of an encrypted forward index update and an inverted index update. Taking the encrypted inverted index update as an example, as shown in Algorithm 7, the updated ciphertext includes operation type (op) and document identity id , but also the MAC of the last ciphertext. In addition, the MAC h stored in DI is updated based on the new ciphertext c_{id} . Algorithm 8 shows the procedures to decrypt the retrieved result of the encrypted inverted index of Jianding. Compared with the decryption algorithm of Hexie, we add the steps to verify the MACs of retrieved ciphertext. Particularly, we start from the last ciphertext to the first ciphertext. If the last retrieved ciphertext matches the locally stored MAC, then we decrypt the ciphertext and update the next checking MAC. If all the ciphertext is successful in the MAC checking, the decrypted result is correct. Otherwise, the decryption is failed. The decryption algorithm for the encrypted forward index is similar. For brevity, we skip the redundant description.

Based on the above construction, there is no need to rebuild the entire MAC chain for modifications in a document because each modification is decomposed into two operations: a deletion and an insertion. This approach ensures that changes are explicitly tracked by first issuing a deletion token and then introducing an insertion token. Additionally, to achieve search result verification, the client only needs to store one additional MAC for each keyword, regardless of the number of documents. The ciphertext stored on the server remains consistent with Hexie.

Algorithm 7: Encrypted Inverted Index Update of Jianding.

Input: (EI_{inx} , DI , (w, id, op))
Output: (EI_{inx} , DI)

- 1: **Client:**
- 2: **if** $DI[w]$ **is empty then**
- 3: $\pi_1 \leftarrow 1^\lambda$
- 4: $h \leftarrow 1^\lambda$
- 5: **else**
- 6: $(\pi_1, h) \leftarrow DI[w]$
- 7: $k_1 || k_2 \leftarrow F(k, w)$
- 8: $c_{id} \leftarrow Enc_2(k_2, op || id || h)$
- 9: $r \leftarrow \{0, 1\}^\lambda$
- 10: $\pi_1 \leftarrow \pi_1 \oplus r$
- 11: $c_w \leftarrow H(\pi_1)$
- 12: $c_a \leftarrow r \oplus PRP(k_1, \pi_1)$
- 13: $h \leftarrow MAC(k_2, c_{id})$
- 14: $DI[w] \leftarrow (\pi_1, h)$
- 15: send ($c_w, (c_a, c_{id})$) to server
- 16: **Server**
- 17: $EI_{inx}[c_w] \leftarrow (c_a, c_{id})$

The computation involves one extra MAC computation and one MAC comparison operation for each retrieved document. However, compared to Hexie, the compaction operation incurs an additional cost due to the need to reconstruct the affected MAC in order to establish a new chain structure with the remaining documents.

Correctness analysis of Jianding: Due to the construction of the ciphertext, the MAC of each ciphertext can be recovered. Any modification conducted on the retrieved ciphertext can be detected. Moreover, the MAC maintains a chain structure. Any of the ciphertexts is lost, resulting in a broken chain structure. For example, if the i th ciphertext is lost, the $i - 1$ th ciphertext loses its MAC and the ciphertext corresponding to the MAC inside $i + 1$ ciphertext is not available. Based on the MAC property and the chain structure, Jianding can verify the search path and result correctly.

V. EFFICIENCY IMPROVEMENT

In this section, following the quality of service management principles in distributed system design [51], we propose a graph-based dictionary sharding algorithm to enhance the efficiency of Hexie and Jianding. This approach considers constraints on the number of servers, server capacity, workload balance, and response time. The first three factors pertain to resource management, while the last factor addresses latency management. We take the encrypted inverted index as an example and explain the details below.

Number of servers constraint: Let $|S|$ be the size of the set of servers and n be the number of keywords. The problem becomes putting n keywords to S servers. The complexity of the problem is $O(|S|^n)$. In order to solve this problem, we use the graph partition algorithm. Particularly, each keyword represents a vertex

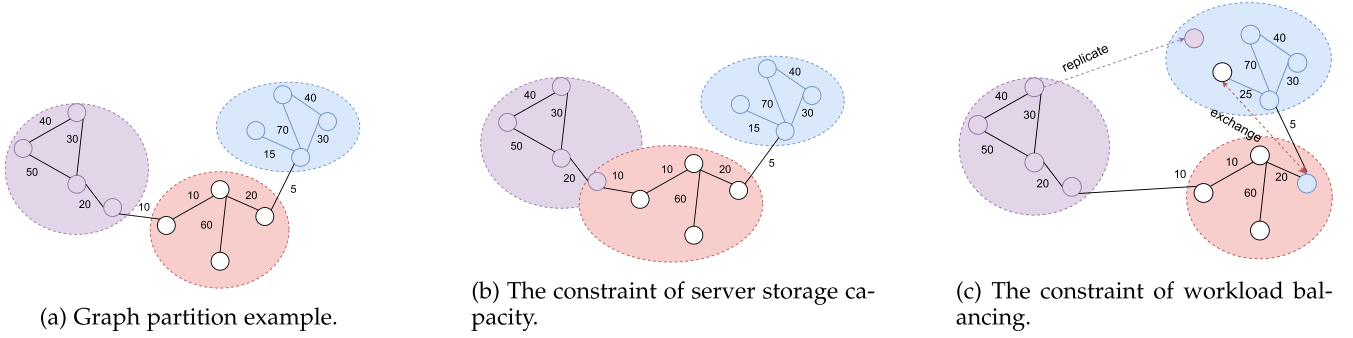


Fig. 1. The proposed graph-based partition algorithm.

Algorithm 8: Decryption of Jianding.

Input: (k_2, w, C, DI)
Output: $(S \text{ or } \text{false})$

- 1: $\pi_1, h \leftarrow DI[w]$
- 2: **if** $C = \phi$ and $\pi_1 \neq \phi$ **then**
- 3: **return** false $\triangleright$ robust against malicious empty search result
- 4: **for all** $i = |C|..1$ **do** $\triangleright$ integrity of search result
- 5: **if** $h = MAC(k_2, C_i)$ **then**
- 6: $op_i, id_i, h_i \leftarrow Dec_2(k_2, C_i)$
- 7: **if** $op_i = 0$ **then**
- 8: $S \leftarrow S \cup id_i$
- 9: **else**
- 10: $S \leftarrow S - id_i$
- 11: $h \leftarrow h_i$
- 12: **else**
- 13: **return** false
- 14: **if** $i = 1$ and $h \neq 1^\lambda$ **then**
- 15: **return** false
- 16: **return** S

and the edge between nodes indicates whether two keywords appear in the same document. The weight of the edge stands for the frequency of two nodes simultaneously appearing in different documents. The problem becomes finding $|S|$ clusters among n nodes. As shown in Fig. 1(a), based on our construction of the graph, we can split n nodes into $|S|$ clusters based on cohesion.

From the above description, we can observe that the keywords that tend to appear synchronously are more likely to be stored on the same server. It results in fewer sessions in batched queries. For example, a client wants to search for ten keywords. If all the keywords are stored in the same server, only one session is required, otherwise, it may reach a maximum of ten sessions. In addition, the partition based on cohesion matches the constraint of the number of servers. However, it is not clear whether it matches the server capacity, workload balance constraint, and response time.

Server capacity constraint: To evaluate the storage requirement of each cluster, we need to collect each node's (keyword) storage requirement. If the total storage demand is higher than the current server's capacity, then the cluster's scale has to be

narrowed. The approach is to recursively cut the farthest node until it matches the capacity requirement. The cut nodes are merged into a nearby cluster. As shown in Fig. 1(b), a node from the purple cluster is cut and merged to the nearby red cluster. The above process can be applied for each cluster until each cluster matches the server capacity requirement.

Workload balance constraint: Given the above setting, we need to evaluate the workload of the clusters. If a cluster has too much workload, we need to balance the workload between clusters. As shown in Fig. 1(c), two approaches exist to balance the workload: node replication and node exchange. If there is enough remaining storage, we can replicate the nodes from the busy cluster to an idle cluster. In such a setting, the workload can be reduced from the busy cluster. If there is no more enough remaining storage, we can replace some nodes of the current cluster with nodes of the idle cluster. To minimize the influence of such replacement, the exchanged nodes should have similar sizes.

Response time constraint: To evaluate the response time, we need to evaluate the query processing cost consisting of processing cost (PC) and transmission cost (TC). PC is related to the number of entries, assuming each entry requires the same time to access. TC is decided by the size of the retrieval data. Given the query q , we are able to compute the query processing cost and evaluate whether it is within the response time constraint R_t . If it exceeds the constraint, we need to narrow the scale of the cluster by using the above approach.

The proposed graph-based dictionary sharding algorithm is not limited to its current scenario and can be applied to other contexts as well. For instance, in disease detection systems, each keyword can represent a type of disease, while the edges illustrate the relationships between diseases. Sharding is then performed based on the proximity of diseases to one another.

VI. PERFORMANCE

In this section, we conduct comprehensive experiments to evaluate the effectiveness of the proposed SSE schemes and dictionary sharding approach. To show the effectiveness of proposed SSE schemes, we implement not only Hexie and Jianding, but also the state-of-the-art DSSE schemes, including FVTSSSE [32], FPPVSSE/FPPVSSE-Ex [31], and a classic DSSE scheme (known as Sophos) [33], for comparison. To evaluate

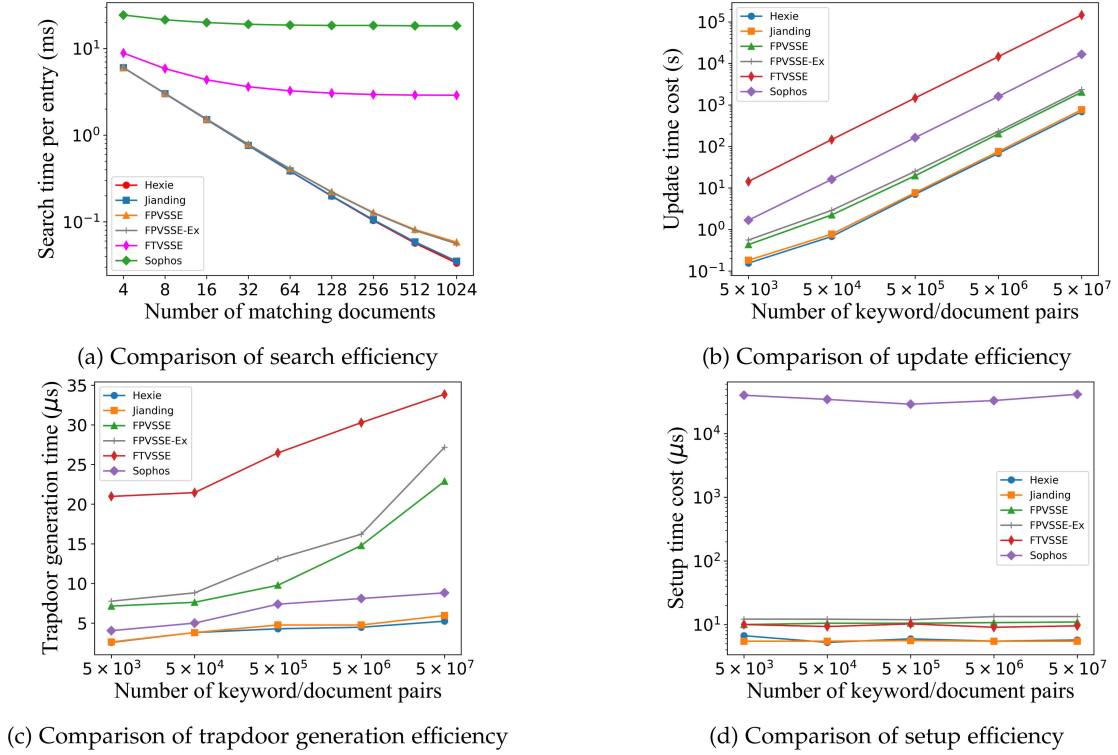


Fig. 2. Performance comparison between the proposed schemes and the state of the art.

TABLE III
CHARACTERISTICS OF SUB-DATASETS

Name	number of entries	number of documents	number of keywords
dataset5e_3	5,000	65	2,490
dataset5e_4	50,000	615	9,906
dataset5e_5	500,000	5,298	44,498
dataset5e_6	5,000,000	44,656	131,092
dataset5e_7	50,000,000	449,287	775,121

the dictionary sharding approach, we focus on the impact of the graph partition algorithm and workload balance.

In our experiments, all the experiments are implemented by python, and all the basic cryptography modules are imported from the pycryptodome library [52]. The well-known public dataset, Enron Email Dataset [53], is chosen as our dataset. It includes 517401 documents and 1666644 keywords in total. We report the average result after running the same operation ten times to avoid burst errors.

A. Evaluation of Hexie and Jianding

For the performance evaluation experiments of Hexie and Jianding, we carried out tests on a desktop computer (Intel Core i7 CPU, 128 GB RAM, 1 TB SSD, Ubuntu 16.04 OS) and Tencent Cloud [54].

To begin with, we randomly choose five datasets based on the size of the inverted index, in particular, the number of entries. The five datasets include 5000, 50000, 500000, 5000000, and 50000000 entries respectively. More details about the five datasets are presented in Table III.

After building the datasets, we start to set the security parameters. In Hexie and Jianding, the size of the cryptographic secret keys is set to 128 bits. In $\sum o \varphi o s$, the symmetric encryption key is set to the same size as Hexie and Jianding, while to be secure the size of the RSA key is set to 2048 bits.

Evaluation of search efficiency: In order to evaluate the search efficiency of the proposed schemes, we randomly select 10 keywords and execute the search algorithm of Hexie, Jianding, FPTSSE, FPTSSE-Ex, FTVSSE, and Sophos over above constructed five datasets. The average time cost of searching a keyword is presented in Fig. 2(a). We observe that Hexie and Jianding exhibit better retrieval efficiency compared to FTVSSE, FPTSSE, FPTSSE-Ex, and Sophos. Additionally, the performance advantage of Hexie and Jianding becomes more pronounced as the number of matching entries increases. Specifically, when the number of matching documents reaches 1024, Hexie and Jianding only require around 0.03 ms to retrieve a matching entry, which is more than $1.6 \times$ faster than FPTSSE and FPTSSE-Ex, $82 \times$ faster than FTVSSE, and $520 \times$ faster than Sophos. The retrieval advantage of Hexie and Jianding is attributed to the elimination of time-consuming cryptographic operations, such as Asymmetric Encryption in Sophos [33], Authenticated Encryption in FPTSSE and FPTSSE-Ex [32], and Puncturable PRFs in FTVSSE [31]. To maintain the same level of security, Hexie and Jianding utilize only HMAC-SHA256 and AES-128 for cryptographic calculations.

Evaluation of update efficiency: In the experiments of evaluating the update algorithm, we first initialized the encrypted dataset to be empty and then update the encrypted dataset by inserting encrypted records one by one. To assess the scalability

of the update algorithm, it was executed over five previously selected datasets. As is shown in Fig. 2(b), the experimental results indicate that the update efficiency of Hexie and Jianding exhibits significant performance advantage than the other schemes. In particular, when the number of entries reaches 5×10^7 , Hexie requires 688 seconds and Jianding requires 769 seconds, outperforming FPTSSE by $2.6\times$, FPTSSE-Ex by $3.0\times$, Sophos by $21.5\times$, and FPTSSE by $190.7\times$.

Evaluation of trapdoor generation efficiency: We also evaluated the time cost of trapdoor generation with various numbers of entries. From the construction of Hexie, Jianding, FPTSSE, FPTSSE-Ex and Sophos, we learn that the trapdoor generation algorithm is quite similar among them. We first randomly select 10 keywords and then execute the trapdoor generation algorithm. The average time cost of generating a trapdoor is reported in Fig. 2(c). Even though Hexie and Jianding exhibit slightly better trapdoor generation efficiency compared to the other schemes, the trapdoor generation overhead of all the schemes is quite similar, ranging from $5.2 \mu s$ to $33.9 \mu s$.

Evaluation of setup efficiency: In this experiment, we evaluate the setup time cost of Hexie, Jianding, FPTSSE, FPTSSE-Ex and Sophos. As shown in Fig. 2(d), the initialization efficiencies of Hexie, Jianding, FPTSSE, FPTSSE-Ex and Sophos are relatively similar, while Sophos demonstrates significantly lower efficiency. This disparity arises because Sophos requires the generation of RSA private and public keys, whereas the other five schemes only need to initialize empty dictionaries and variables.

B. Evaluation of Dictionary Sharding Solution

In this section, we evaluate the significance of the dictionary sharding approach based on Jianding. In order to simulate multiple servers, we install a VMware Workstation Pro [55] to simulate servers. In addition, we use the open-source library Metis [56] to implement our graph partition algorithm.

In respect of the dictionary sharding evaluation, we devise three experiments. The first experiment is applied to show that Jianding equipped with a dictionary with the proposed sharding mechanism has better performance than Jianding applying a dictionary without a sharding mechanism. The second experiment is designed to demonstrate that a balanced workload is an essential factor of dictionary sharding. The third experiment is planned to show the impact of the keyword selection algorithm on the performance of the proposed dictionary sharding scheme.

Evaluation of the impact of dictionary sharding: In this experiment, we evaluate the performance of Jianding with the proposed graph-based sharding algorithm. In particular, we first map the dictionary into a graph and then split the graph into two parts. After that, we randomly select 20000 keywords and launch 20000 concurrent queries to the servers. For comparison, with the same keywords, we also launch 20000 concurrent search requests to the server without a dictionary sharding algorithm. To show the scalability, the above search operation is executed in three datasets containing 50, 500, and 5000 documents respectively. The average keyword search time is presented in Fig. 3. From the figure, we can learn that compared with the dictionary

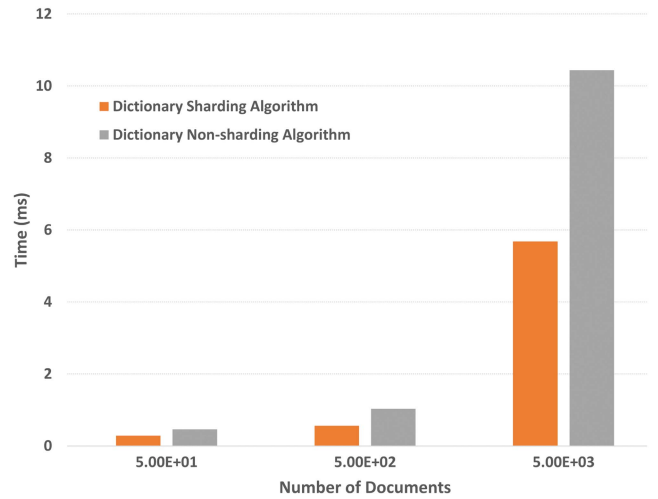


Fig. 3. Evaluation of the impact of dictionary sharding.

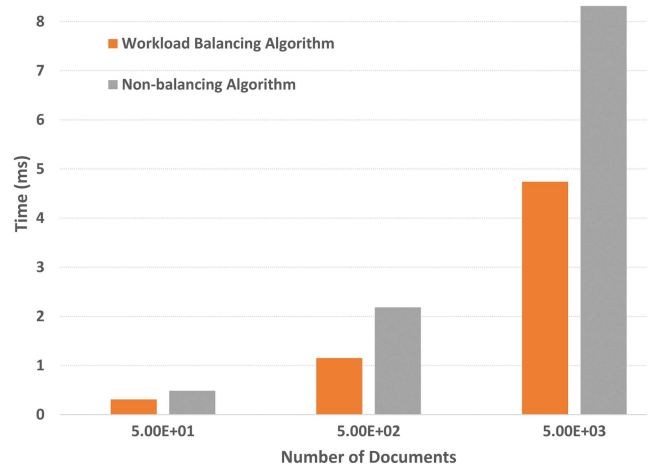


Fig. 4. Evaluation of the impact of workload balance.

non-sharding algorithm, Jianding with the proposed dictionary sharding algorithm has approximately two times faster than Jianding without the dictionary sharding algorithm.

Evaluation of the impact of workload balance: In order to show the effectiveness of the workload balancing strategy, we design two schemes based on Jianding, one with a workload balance algorithm and the other one without that algorithm. Based on the randomly selected 20000 keywords, we launch 20000 concurrent search requests to two servers. In the workload balance scheme, We move several high-frequent keywords from a high-workload server to a low-workload server so that both servers reach similar workloads. In the other scheme, we do not execute that transfer. Fig. 4 shows that Jianding with the workload balancing scheme consumes much less time to complete the keyword search operation.

Evaluation of the impact of the keywords selection: In the above experiment, the keywords are randomly selected. In this experiment, to simulate human behavior better [57], [58], instead of random selection, we get all the keywords by using the random walk algorithm [59] by specifying a random initial node. To evaluate the search efficiency, we first select 20000 keywords

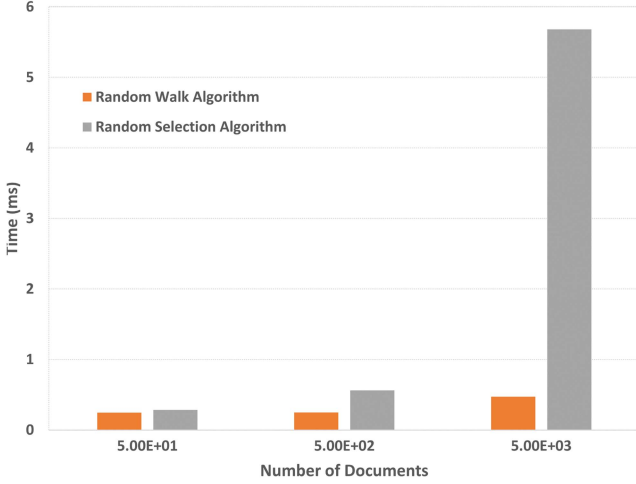


Fig. 5. Evaluation of the impact of keywords selection.

by using the random walk algorithm. Based on the selected keywords, we launch 20000 concurrent search queries to the servers. As a comparison, we also launch the same queries to the servers with 20000 randomly selected keywords. To show the scalability of the proposed approach, we also run the above operation over three datasets containing 50, 500, and 5000 documents respectively. The average keyword search time is demonstrated in Fig. 5. From the figure, we can observe that the search time required by the keywords selected by the random walk algorithm is much less compared with the search time consumed by the randomly selected keywords. After analyzing the data, we learn that searching for neighboring keywords significantly reduces the time of establishing connections between the client and servers.

VII. SECURITY ANALYSIS

The paper focuses solely on searching over forward and inverted indices rather than querying documents directly. Since the principles used to build both indices are the same, the search processes are identical. Therefore, due to space limitations, we provide a detailed security analysis only for the keyword-based search over the inverted index. In this section, we first present the leakage function and then describe the forward and backward privacy following [10].

The privacy of a searchable encryption scheme is parameterized by a leakage function $\mathcal{L} = (\mathcal{L}^{stp}, \mathcal{L}^{srch}, \mathcal{L}^{updt})$. $\mathcal{L}^{stp}$ refers to leakage during setup, $\mathcal{L}^{srch}$ during a search operation, and $\mathcal{L}^{updt}$ during updates. To further define forward and backward privacy, we define the $sp(w)$, $TimeDB(w)$ and $DelHist(w)$ as below.

$$sp(w) = \{t | (w, t) \in Q\}$$

$$TimeDB(w) = \{(t, id) | (t, add, (w, id)) \in Q$$

$$\text{and } \forall t', (t', del, (w, id)) \notin Q\}$$

$$Updates(w) = \{t | (t, add, (w, id)) \in Q$$

$$\text{or } (t, del, (w, id)) \in Q\}$$

$$DelHist(w) = \{(t_{add}, t_{del}) | \exists id, (t_{add}, add, (w, id)) \in Q \\ \text{add } (t_{del}, del, (w, id)) \in Q\}$$

For a keyword w , $sp(w)$ returns the timestamps at which keyword w is searched, $TimeDB(w)$ returns all timestamp/file-identifier pairs of the keyword w , where all the added files have not been deleted, $Updates(w)$ returns the timestamps of updates on w , and $DelHist(w)$ returns all the insertion/deletion timestamp pairs referring to the keyword w if there is a file identifier id such that (w, id) has been added to EI and subsequently deleted from EI .

Definition 4 (Forward privacy): A $\mathcal{L}$ -adaptively-secure SSE is forward-private if the update leakage function $\mathcal{L}^{updt}$ can be written as: $\mathcal{L}^{updt}(op, w, id) = \mathcal{L}'(op, id)$, where $\mathcal{L}'$ is a stateless function.

For the proposed scheme, we achieve $\mathcal{L}^{updt}(op, w, id) = \phi$ and $\mathcal{L}^{stp}(\lambda) = \phi$.

Definition 5 (Backward privacy I): A $\mathcal{L}$ -adaptively secure SSE is Type-I backward secure iff the search and update leakage function $\mathcal{L}^{updt}$ and $\mathcal{L}^{srch}$ can be written as $\mathcal{L}^{updt}(op, w, id) = \mathcal{L}'(op)$, $\mathcal{L}^{srch}(w) = \mathcal{L}''(TimeDB(w), a_w)$, where a_w is the total number of updates, $\mathcal{L}'$ and $\mathcal{L}''$ are stateless.

Definition 6 (Backward privacy II): A $\mathcal{L}$ -adaptively-secure SSE is Type-II backward secure iff the search and update leakage function can be written as $\mathcal{L}^{updt}(op, w, id) = \mathcal{L}'(op, w)$, $\mathcal{L}^{srch}(w) = \mathcal{L}''(sp(w), TimeDB(w), Updates(w))$, where $\mathcal{L}'$ and $\mathcal{L}''$ are stateless.

Definition 7 (Backward privacy III): A $\mathcal{L}$ -adaptively-secure SSE is Type-III backward secure iff the search and update leakage function can be written as $\mathcal{L}^{updt}(op, w, id) = \mathcal{L}'(op, w)$, $\mathcal{L}^{srch}(w) = \mathcal{L}''(sp(w), TimeDB(w), DelHist(w))$, where $\mathcal{L}'$ and $\mathcal{L}''$ are stateless.

1) Security Definition: The security definition of DSSE is derived from two games: $DSSEReal_{\mathcal{A}}^{\Gamma}(1^\lambda)$ and $DSSEReal_{\mathcal{A},S}^{\Gamma}(1^\lambda)$. The game $DSSEReal_{\mathcal{A}}^{\Gamma}(1^\lambda)$ is executed using DSSE. The game $DSSEReal_{\mathcal{A},S}^{\Gamma}(1^\lambda)$ is simulated using the leakage of DSSE. The leakage can be described by a function $\mathcal{L} = (\mathcal{L}^{stp}, \mathcal{L}^{srch}, \mathcal{L}^{updt})$, which describes what information is leaked to the adversary $\mathcal{A}$. Using the leakage function $\mathcal{L}$ as a measure of information leakage, if adversary $\mathcal{A}$ cannot discern these two games, then the information that can be inferred from it is the only information leaked. In more formal terms,

- $DSSEReal_{\mathcal{A}}^{\Gamma}(1^\lambda)$: On input a database $\mathbb{DB}$, which is chosen by the adversary $\mathcal{A}$, it outputs $\mathbb{EDB}$ by using $Setup(1^\lambda, \mathbb{DB})$ to the adversary $\mathcal{A}$. $\mathcal{A}$ can repeatedly perform a search query q (or an update query (op, ind)). The game outputs the results generated by running $Search(q)$ (or $Update(op, ind)$) to the adversary $\mathcal{A}$. Eventually, $\mathcal{A}$ outputs a bit.
- $DSSEReal_{\mathcal{A},S}^{\Gamma}(1^\lambda)$: On input a database which is chosen by the adversary $\mathcal{A}$, it outputs $\mathbb{EDB}$ to the adversary $\mathcal{A}$ by using a simulator $\mathcal{S}(\mathcal{L}^{stp})$. Then, it simulates the results for the search query q by using the simulator $\mathcal{S}(\mathcal{L}^{srch}(q))$ and uses $\mathcal{S}(\mathcal{L}^{updt}(op, ind))$ to simulate the results for update query (op, ind) . Eventually, $\mathcal{A}$ outputs a bit.

Definition 8: A DSSE scheme Γ is $\mathcal{L}$ -adaptively-secure if for every PPT adversary $\mathcal{A}$, there exists an efficient simulator $\mathcal{S}$ such that $|\Pr[\text{DSSEReal}_{\mathcal{A}}^{\Gamma}(1^{\lambda}) = 1] - \Pr[\text{DSSEIdeal}_{\mathcal{A},\mathcal{S}}^{\Gamma}(1^{\lambda}) = 1]| \leq \text{negl}(1^{\lambda})$.

2) *Security Analysis of Hexie:*

Definition 9 ($\mathcal{L}$ -adaptively-secure DSSE game): Let Π be the Hexie scheme, $\mathcal{L}$ be the leakage function, $\mathcal{A}$ be a PPT adversary, and $\mathcal{S}$ be a PPT simulator. The $\text{Real}_{\Pi,\mathcal{A}}(\lambda)$ and $\text{Ideal}_{\Pi,\mathcal{A},\mathcal{L}}(\lambda)$ experiments of $\mathcal{L}$ -adaptively-secure DSSE game are defined as below.

Real $_{\Pi,\mathcal{A}}(\lambda)$
 $(DI, DF, EI_{inx}, EF_{inx}, k) \leftarrow \text{Setup}(1^{\lambda}, \mathbb{DB})$
 $\text{transcript} \leftarrow (EI_{inx}, EF_{inx})$
 for $k = 1$ to q do
 $Q_k = (\text{type}_k, \text{param}_k) \leftarrow \mathcal{A}(\text{transcript})$
 if $\text{type}_k = \text{Update}$ then
 $R_k \leftarrow \text{Update}(\text{param}_k, EI_{inx}, EF_{inx})$
 else
 $R_k \leftarrow \text{Search}(\text{param}_k, EI_{inx}, EF_{inx})$
 Append (Q_k, R_k) to transcript
 $b \leftarrow \mathcal{A}(\text{transcript})$
 return b

Ideal $_{\Pi,\mathcal{A},\mathcal{L}}(\lambda)$
 $(DI, DF, EI_{inx}, EF_{inx}, k) \leftarrow \mathcal{S}(\mathcal{L}^{stp}, \mathbb{DB})$
 $\text{transcript} \leftarrow (EI_{inx}, EF_{inx})$
 for $k = 1$ to q do
 $Q_k = (\text{type}_k, \text{param}_k) \leftarrow \mathcal{A}(\text{transcript})$
 if $\text{type}_k = \text{Update}$ then
 $R_k \leftarrow \mathcal{S}(\mathcal{L}^{updt}(\text{param}_k))$
 else
 $R_k \leftarrow \mathcal{S}(\mathcal{L}^{srch}(\text{param}_k))$
 Append (Q_k, R_k) to transcript
 $b \leftarrow \mathcal{A}(\text{transcript})$
 return b

We say Hexie scheme is a $\mathcal{L}$ -adaptively-secure DSSE if $|\Pr[\text{Real}_{\Pi,\mathcal{A}}(1^{\lambda}) = 1] - \Pr[\text{Ideal}_{\Pi,\mathcal{A},\mathcal{L}}(1^{\lambda}) = 1]| \leq \text{negl}(1^{\lambda})$.

Theorem 1: Let PRP be a pseudorandom permutation, F be a pseudorandom function, Enc_2 be a semantically symmetric encryption algorithm, H be modelled as a programmable random oracle, and λ be a security parameter. If $\mathcal{L}_{Hexie} = (\mathcal{L}_{Hexie}^{srch}, \mathcal{L}_{Hexie}^{updt})$ where $\mathcal{L}_{Hexie}^{srch} = (\text{sp}(w), \text{TimeDB}(w), \text{Updates}(w))$, $\mathcal{L}_{Hexie}^{updt} = \phi$, then Π_{Hexie} achieves $\mathcal{L}_{Hexie}$ -adaptively-secure (backward privacy II).

Proof: The proof proceeds using a hybrid argument, starting from the real-world game. For brevity, in the proof, we only focus on the inverted index. The security of the forward index can be proved similarly.

Game G_0 : this game is exactly the same as the real-world execution. Then we have (1).

$$\Pr[\text{Real}_{\Pi,\mathcal{A}}(1^{\lambda}) = 1] = \Pr[G_0(1^{\lambda}) = 1]. \quad (1)$$

Game G_1 : this game is same as G_0 except that we replace the call to the F by picking a random output every time for a previously unseen keyword and storing it in a table T_1 . For repeated keywords, refers to the table. The advantage for the adversary to distinguish G_1 from G_0 is the same as the advantage

for a distinguisher D to distinguish the output of F from a random string. It derives (2).

$$\Pr[G_0(1^{\lambda}) = 1] - \Pr[G_1(1^{\lambda}) = 1] \leq \text{Adv}_{F,D}^{prf}(\lambda). \quad (2)$$

Game G_2 : this game is same as G_1 except that the encrypted value from running the encryption algorithm is randomly selected and stored into a mapping table T_2 . The advantage for the adversary to distinguish G_2 from G_1 is the same as the advantage for a distinguisher D to distinguish the output of Enc_2 from a random string. It derives (3).

$$\Pr[G_1(1^{\lambda}) = 1] - \Pr[G_2(1^{\lambda}) = 1] \leq \text{Adv}_{Hexie,D}^{Enc_2}(\lambda) \quad (3)$$

Game G_3 : this game is same as G_2 except that the PRP algorithm is replaced by uniformly selecting a value from the output of PRP algorithm for each unseen input and it is stored into a table T_3 . Then we have (4).

$$\Pr[G_2(1^{\lambda}) = 1] - \Pr[G_3(1^{\lambda}) = 1] \leq \text{Adv}_{PRP,D}^{PRP}(\lambda) \quad (4)$$

Game G_4 : This game is same as G_3 except that the call to H is replaced by a programmable random oracle. If the input has not been queried before, a uniformly random value is selected from the output space of H . The pair of input π_1 and output c_w are stored into the table T_4 . Base on the given table T_4 and the oracle H is reprogrammed. Note that if a new input with a same existing value stored in T_4 , the program aborts. Since each input value stored in T_4 is randomly chosen by the challenger, the adversary guesses the value correctly with probability $\frac{qN^*}{2^{\lambda}}$ for the maximum q queries and N^* distinct values in the table. Therefore, we have (5).

$$\Pr[G_3(1^{\lambda}) = 1] - \Pr[G_4(1^{\lambda}) = 1] \leq \frac{qN^*}{2^{\lambda}} \quad (5)$$

Simulator: We are ready to show that G_4 and Simulator are indistinguishable. The simulator $\mathcal{S}$ is built only with the leakage function $\mathcal{L} = (\mathcal{L}^{stp}, \mathcal{L}^{srch}, \mathcal{L}^{updt})$ as input. The details of each leakage function are outlined as follows.

- 1) $\mathcal{L}^{stp}(\lambda) = \phi$
- 2) $\mathcal{L}^{updt} = \phi$
- 3) $\mathcal{L}^{srch}(EI_{idx}, \text{trp}) = (\text{sp}(w), \text{TimeDB}(w), \text{Update}(w))$

The Setup algorithm can be simulated as shown in Algorithm 9. During this phase, simulator $\mathcal{S}$ takes the leakage function $\mathcal{L}^{stp}(\lambda)$ as input, initializes two empty list, DI and EI_{inx} , and a timestamp variable t . It then sends EI_{inx} to the server. It is clear that the $\mathcal{S}.\text{Setup}(\mathcal{L}^{stp}(\lambda))$ is indistinguishable from the real Hexie.Setup algorithm, as simulated EI_{inx} is identical to the real list.

The Update algorithm is simulated as described in Algorithm 10. When an adversary $\mathcal{A}$ issues an update query $(op, (w, id))$, the simulator $\mathcal{S}$ uses the leakage function $\mathcal{L}^{updt}(op, (w, id))$ as input, computes the timestamp t for the update query, randomly selects a ciphertext, and sends it to the server. Due to the randomness of the oracle H and pseudorandom permutation PRP , and the semantically secure encryption algorithm Enc_2 , the simulated ciphertext has the same distribution as the real ciphertext generated by Hexie.Update in the RO model. Consequently, $\mathcal{S}.\text{Update}(\mathcal{L}^{updt}(op, (w, id)))$ is indistinguishable from the real Hexie.Update algorithm in Game G_4 .

Algorithm 9: $\mathcal{S}.\text{Setup}(\mathcal{L}^{stp}(\lambda))$.

- 1: initialise a list DI to store local data for inverted index
- 2: initialise a list EI_{inx} to store encrypted data of inverted index
- 3: initialise the variable $t \leftarrow 0$.
- 4: send EI_{inx} to the server

Algorithm 10: $\mathcal{S}.\text{Update}(\mathcal{L}^{updt}(op, (w, id)))$.

- 1: Client:
- 2: $t \leftarrow t + 1$
- 3: $c_w[t] \leftarrow \{0, 1\}^\lambda$
- 4: $c_a[t] \leftarrow \{0, 1\}^\lambda$
- 5: $c_{id}[t] \leftarrow \{0, 1\}^{|Enc_2(\cdot, \cdot)|}$
- 6: send $(c_w[t], (c_a[t], c_{id}[t]))$ to server

The Search algorithm is simulated as described in Algorithm 11. In the simulation, the primary focus is on constructing the adversary's view, while the ciphertext of document identifiers is omitted, as it does not impact the adversary's view in the experiment. When an adversary $\mathcal{A}$ issues a search query w , simulator $\mathcal{S}$ utilizes leakage function $\mathcal{L}^{srch}(w)$ as input and proceeds with the following steps:

- 1) The simulator $\mathcal{S}$ computes the timestamp t of the search query and extracts the first timestamp $\bar{w}$ associated with the keyword w . Using this timestamp, the corresponding key is retrieved from the table T_1 . Meanwhile, the state information π_i is fetched from list DI .
- 2) Extract all timestamps $t_0, \dots, t_n$ from $\text{Updates}(w)$. If $n = 0$ and $t_0 = 0$, return an empty result.
- 3) For all timestamps from t_n to t_0 , program the oracle H and the pseudorandom function PRP .
- 4) Finally send the trapdoor to the server.

In the above steps, upon receiving the search trapdoor from simulator $\mathcal{S}$, the server executes the actual search process to retrieve all matching ciphertexts, with the hash function H acting as a random oracle and PRP implemented via table T_3 . Hence, $\mathcal{S}.\text{Search}(\mathcal{L}^{srch}(w))$ is indistinguishable from $\text{Hexie}.\text{Search}$ algorithm in Game G_4 .

From the above analysis, the above simulator $\mathcal{S}$ utilizes leakage functions as inputs to simulate an ideal game of Hexie, which remains indistinguishable from a real game of Hexie under the adaptive attacks in the RO model. Therefore, we have (6).

$$\begin{aligned} & \Pr[\mathbf{Real}_{\pi, \mathcal{A}}(1^\lambda) = 1] - \Pr[\mathbf{Ideal}_{\pi, \mathcal{A}, \mathcal{L}}(1^\lambda) = 1] \\ & \leq \text{Adv}_{F, D}^{prf}(\lambda) + \text{Adv}_{Hexie, D}^{Enc_2}(\lambda) + \text{Adv}_{PRP, D}^{PRP}(\lambda) + \frac{qN^*}{2^\lambda}. \end{aligned} \quad (6)$$

□

A. Security Analysis of Jianding

From the construction of Jianding, we can learn that the added MAC can be considered as part of the document identity, which does not weaken the security. Therefore, it is capable to achieve the same security as Hexie.

Algorithm 11: $\mathcal{S}.\text{Search}(\mathcal{L}^{srch}(w))$.

- 1: Client:
- 2: $t \leftarrow t + 1$
- 3: $\bar{w} \leftarrow \text{minsp}(w)$
- 4: $k_{1, \bar{w}} || k_{2, \bar{w}} \leftarrow T_1[\bar{w}]$
- 5: $\pi_t \leftarrow DI[\bar{w}]$
- 6: Extract all timestamps $t_0, \dots, t_n$ from $\text{Updates}(w)$ where $t_i < t_j$ if $i < j$
- 7: **if** $n = 0$ and $t_0 = 0$ **then**
- 8: **return** ϕ
- 9: **for all** $i = n$ **to** 1 **do**
- 10: $\pi_{i-1} \leftarrow \{0, 1\}^\lambda$
- 11: $T_4[\pi_{i-1}] \leftarrow c_w[t_i]$
- 12: $r \leftarrow \pi_i \oplus \pi_{i-1}$
- 13: $T_3[k_{1, \bar{w}}] \leftarrow r \oplus c_a[t_i]$
- 14: send trapdoor $(k_{1, \bar{w}}, \pi_t)$ to the server

VIII. CONCLUSION

In this paper, to the best of our knowledge, we propose the first SSE scheme capable to support dynamic datasets with forward and backward privacy, integrity verification of non-empty and empty search results, efficient search, non-interactive, light client, and both forward and backward index. To support dynamic dataset, we first propose Hexie SSE scheme. It achieves not only forward privacy and backward privacy, but also efficient search, non-interactive, and light client. To be robust against empty, incomplete, or incorrect search results, we propose the Jianding SSE scheme. Moreover, we propose an approach to enhance search efficiency via dictionary sharding. Finally, we conduct comprehensive experiments to evaluate the effectiveness of the proposed schemes.

REFERENCES

- [1] N. Aaraj, C. Marcolla, and X. Zhu, "Exipnos: An efficient verifiable dynamic symmetric searchable encryption scheme with forward and backward privacy," in *Proc. Int. Conf. Cryptol. India*, 2021, pp. 487–509.
- [2] X. Zhu, E. Ayday, R. Vitenberg, and N. R. Veeraragavan, "Privacy-preserving search for a similar genomic makeup in the cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2771–2788, Jul./Aug. 2022.
- [3] X. Zhu, E. Ayday, and R. Vitenberg, "A privacy-preserving framework for outsourcing location-based services to the cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 1, pp. 384–399, Jan./Feb. 2021.
- [4] T. Li, Y. Tian, J. Xiong, and M. Z. Bhuiyan, "FVP-EOC: Fair, verifiable and privacy-preserving edge outsourcing computing in 5G-enabled IIoT," *IEEE Trans. Ind. Inform.*, vol. 19, no. 1, pp. 940–950, Jan. 2023.
- [5] Millions of Mexican voter records 'were accessible online', 2016. [Online]. Available: <https://www.bbc.com/news/technology-36128745>
- [6] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proc. 2000 IEEE Symp. Secur. Privacy*, 2000, pp. 44–55.
- [7] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," *J. Comput. Secur.*, vol. 19, no. 5, pp. 895–934, 2011.
- [8] S. Kamara, C. Papamanthou, and T. Roeder, "Dynamic searchable symmetric encryption," in *Proc. 2012 ACM Conf. Comput. Commun. Secur.*, 2012, pp. 965–976.
- [9] D. Cash, S. Jarecki, C. Jutla, H. Krawczyk, M.-C. Roşu, and M. Steiner, "Highly-scalable searchable symmetric encryption with support for Boolean queries," in *Proc. Annu. Cryptol. Conf., Springer*, 2013, pp. 353–373.

- [10] R. Bost, B. Minaud, and O. Ohrimenko, "Forward and backward private searchable encryption from constrained cryptographic primitives," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1465–1482.
- [11] W. Sun, X. Liu, W. Lou, Y. T. Hou, and H. Li, "Catch you if you lie to me: Efficient verifiable conjunctive keyword search over large dynamic encrypted cloud data," in *Proc. 2015 IEEE Conf. Comput. Commun.*, 2015, pp. 2110–2118.
- [12] A. Juels and B. S. Kaliski Jr., "Pors: Proofs of retrievability for large files," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 584–597.
- [13] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, "Enabling public auditability and data dynamics for storage security in cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 5, pp. 847–859, May 2011.
- [14] E. Stefanov, M. van Dijk, A. Juels, and A. Oprea, "Iris: A scalable cloud file system with efficient integrity checks," in *Proc. 28th Annu. Comput. Secur. Appl. Conf.*, 2012, pp. 229–238.
- [15] Y. Zhu, H. Hu, G.-J. Ahn, and M. Yu, "Cooperative provable data possession for integrity verification in multicloud storage," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 12, pp. 2231–2244, Dec. 2012.
- [16] Q. Chai and G. Gong, "Verifiable symmetric searchable encryption for semi-honest-but-curious cloud servers," in *Proc. 2012 IEEE Int. Conf. Commun.*, 2012, pp. 917–922.
- [17] K. Kurosawa and Y. Ohtaki, "UC-secure searchable symmetric encryption," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, Springer, 2012, pp. 285–298.
- [18] X. Jiang, J. Yu, J. Yan, and R. Hao, "Enabling efficient and verifiable multi-keyword ranked search over encrypted cloud data," *Inf. Sci.*, vol. 403, pp. 22–41, 2017.
- [19] X. Zhu, Q. Liu, and G. Wang, "A novel verifiable and dynamic fuzzy keyword search scheme over encrypted data in cloud computing," in *Proc. 2016 IEEE Trustcom/BigDataSE/ISPA*, 2016, pp. 845–851.
- [20] Y.-C. Chang and M. Mitzenmacher, "Privacy preserving keyword searches on remote encrypted data," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, Springer, 2005, pp. 442–455.
- [21] E. Stefanov, C. Papamanthou, and E. Shi, "Practical dynamic searchable encryption with small leakage," in *Proc. NDSS*, 2014, pp. 1–15.
- [22] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. Conf. Theory Appl. Cryptographic Techn.*, Springer, 1987, pp. 369–378.
- [23] Message Authentication Code, 2022. [Online]. Available: https://en.wikipedia.org/wiki/Message_authentication_code#cite_note-5
- [24] J. Benaloh and M. D. Mare, "One-way accumulators: A decentralized alternative to digital signatures," in *Proc. Workshop Theory Appl. Cryptographic Techn.*, Springer, 1993, pp. 274–285.
- [25] C. Chen et al., "An efficient privacy-preserving ranked keyword search method," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 4, pp. 951–963, Apr. 2016.
- [26] K. Kurosawa and Y. Ohtaki, "How to update documents verifiably in searchable symmetric encryption," in *Proc. Int. Conf. Cryptol. Netw. Secur.*, Springer, 2013, pp. 309–328.
- [27] E.-J. Goh, "Secure indexes," *Cryptol. ePrint Arch.*, Rep. 2003/216, 2003. [Online]. Available: <http://eprint.iacr.org/2003/216/>
- [28] S.-F. Sun et al., "Practical non-interactive searchable encryption with forward and backward privacy," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, 1–18.
- [29] S.-F. Sun et al., "Practical backward-secure searchable encryption from symmetric puncturable encryption," in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 763–780.
- [30] J. Ghareh Chamani, D. Papadopoulos, C. Papamanthou, and R. Jalili, "New constructions for forward and backward private symmetric searchable encryption," in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1038–1055.
- [31] C. Guo, W. Li, X. Tang, K.-K. R. Choo, and Y. Liu, "Forward private verifiable dynamic searchable symmetric encryption with efficient conjunctive query," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 746–763, Mar./Apr. 2024.
- [32] D. Yuan, S. Cui, and G. Russello, "We can make mistakes: Fault-tolerant forward private verifiable dynamic searchable symmetric encryption," in *Proc. IEEE 7th Eur. Symp. Secur. Privacy*, 2022, pp. 587–605.
- [33] R. Bost, "Σoφoc: Forward secure searchable encryption," in *Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur.*, E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds, Vienna, Austria, Oct. 24–28, 2016, pp. 1143–1154, doi: [10.1145/2976749.2978303](https://doi.org/10.1145/2976749.2978303).
- [34] R. Bost, P.-A. Fouque, and D. Pointcheval, "Verifiable dynamic symmetric searchable encryption: Optimality and forward security," *IACR Cryptol. ePrint Arch.*, Rep. 2016/62, 2016.
- [35] J. Zhu, Q. Li, C. Wang, X. Yuan, Q. Wang, and K. Ren, "Enabling generic, verifiable, and secure data search in cloud services," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 8, pp. 1721–1735, Aug. 2018.
- [36] Z. Zhang, J. Wang, Y. Wang, Y. Su, and X. Chen, "Towards efficient verifiable forward secure searchable symmetric encryption," in *Proc. Eur. Symp. Res. Comput. Secur.*, Springer, 2019, pp. 304–321.
- [37] X. Ge et al., "Towards achieving keyword search over dynamic encrypted cloud data with symmetric-key based verification," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 1, pp. 490–504, Jan./Feb. 2021.
- [38] I. Demertzis, J. G. Chamani, D. Papadopoulos, and C. Papamanthou, "Dynamic searchable encryption with small client storage," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, p. 1227.
- [39] N. Cao, C. Wang, M. Li, K. Ren, and W. Lou, "Privacy-preserving multi-keyword ranked search over encrypted cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 1, pp. 222–233, Jan. 2014.
- [40] J. Wang et al., "Efficient verifiable fuzzy keyword search over encrypted data in cloud computing," *Comput. Sci. Inf. Syst.*, vol. 10, no. 2, pp. 667–684, 2013.
- [41] W. Sun et al., "Privacy-preserving multi-keyword text search in the cloud supporting similarity-based ranking," in *Proc. 8th ACM SIGSAC Symp. Inf. Comput. Commun. Secur.*, 2013, pp. 71–82.
- [42] Z. Fu, L. Xia, X. Sun, A. X. Liu, and G. Xie, "Semantic-aware searching over encrypted data for cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 9, pp. 2359–2371, Sep. 2018.
- [43] S. Kamara and C. Papamanthou, "Parallel and dynamic searchable symmetric encryption," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, Springer, 2013, pp. 258–274.
- [44] D. Cash et al., "Dynamic searchable encryption in very-large databases: Data structures and implementation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014, vol. 14, pp. 1–16.
- [45] M. Naveed, M. Prabhakaran, and C. A. Gunter, "Dynamic searchable encryption via blind storage," in *Proc. 2014 IEEE Symp. Secur. Privacy*, 2014, pp. 639–654.
- [46] Z. Xia, X. Wang, X. Sun, and Q. Wang, "A secure and dynamic multi-keyword ranked search scheme over encrypted cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 2, pp. 340–352, Feb. 2016.
- [47] C. Guo, X. Chen, Y. Jie, Z. Fu, M. Li, and B. Feng, "Dynamic multi-phrase ranked search over encrypted data with symmetric searchable encryption," *IEEE Trans. Services Comput.*, vol. 13, no. 6, pp. 1034–1044, Nov./Dec. 2020.
- [48] Inverted Index. Accessed: May 12, 2022. [Online]. Available: <https://linux.nist.gov/dads/HTML/invertedIndex.html>
- [49] Secret Sharing. Accessed: May 10, 2022. [Online]. Available: https://en.wikipedia.org/wiki/Secret_sharing
- [50] T-out-of-N secret sharing scheme. Accessed: May 10, 2022. [Online]. Available: <https://www.cs.columbia.edu/tal/4261/F19/secretsharingf19.pdf>
- [51] G. Coulouris, J. Dollimore, and T. Kindberg, *Distributed systems: Concepts and design* edition 3, 2001.
- [52] Python pycryptodome library, 2022. [Online]. Available: <https://pypi.org/project/pycryptodome/>
- [53] W. W. Cohen, "Enron email dataset," 2015. [Online]. Available: <https://www.cs.cmu.edu/~enron/>
- [54] Tencent, "Tencent cloud service," 2024. [Online]. Available: <https://cloud.tencent.com/>
- [55] VMware Workstation Player, Accessed: Dec 10, 2022. [Online]. Available: <https://www.vmware.com/products/workstation-player.html>
- [56] K. Lab, "Metis, serial graph partitioning and fill-reducing matrix ordering," 2022. [Online]. Available: <http://glaros.dtc.umn.edu/gkhome/metis/metis/download/>
- [57] H. Namazi and V. V. Kulish, "Fractional diffusion based modelling and prediction of human brain response to external stimuli," *Comput. Math. Methods Med.*, vol. 2015, 2015, Art. no. 148534.
- [58] S. C. Ahearn, S. Dodge, A. Simcharoen, G. Xavier, and J. L. Smith, "A context-sensitive correlated random walk: A new simulation model for movement," *Int. J. Geographical Inf. Sci.*, vol. 31, no. 5, pp. 867–883, 2017.
- [59] Random walks in graphs. Accessed: Dec 10, 2022. [Online]. Available: <https://people.csail.mit.edu/madry/gems/notes/lecture9.pdf>



Xiaojie Zhu (Senior Member, IEEE) received the MS degree from the University of Chinese Academy of Sciences in 2015 and the PhD degree from the University of Oslo, in 2021. He is a staff research scientist with KAUST. His research focuses on cloud security, applied cryptography, and distributed systems.



Peisong Shen received the bachelor's degree from the University of Science and Technology of China in 2012 and the graduated degree from the University of Chinese Academy of Sciences, in 2018. He is an assistant researcher with the Institute of Information Engineering, Chinese Academy of Sciences. His research interest includes applied cryptography and data privacy protection. Since then, he joined the Institute of Information Engineering as a research assistant.



Jiancong Zhou received the BEng degree in computer science from Jilin University, Changchun, China, in 2010 and the MS degree in software engineering from Beijing Jiaotong University, Beijing, China, in 2015. He is currently working toward the PhD degree with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing. His research interest is symmetric searchable encryption.



cybersecurity, information privacy, automotive cyber security, and blockchain.

Shabnam Kasra Kermanshahi received the PhD degree in security sciences, with a focus on applied cryptography, from Monash University in Melbourne, Australia. She is a senior lecturer specializing in cybersecurity with the University of New South Wales (UNSW) Canberra. She has extensive experience in the field of cybersecurity, having served as a postdoctoral research fellow with CSIRO's Data61, a lecturer with RMIT University's School of Computing Technologies, and a senior lecturer with UNSW Canberra. Her research interests lie in the areas of cryptography,



Yueyue Dai (Member, IEEE) is an associate professor with the Huazhong University of Science and Technology. She was a researcher with Nanyang Technological University, Singapore, in 2020. Her current research interests include edge intelligence, the internet of vehicles and blockchain. She serves/has served as the guest editor for many leading journals *IEEE Network*, *Future Generation Computer Systems*, and *Digital Communications and Networks*, PC member for IEEE Symposium on Blockchain, TPC member for IEEE ICC 2022, IEEE GLOBECOM 2021, IEEE ICC 2021.



on Computer Science and its Applications. He is serving as an associate editor of the following journals: *Journal of Security and Communication Networks*, Wiley; *IEEE Transactions on Information Forensics and Security* (senior area editor); *KSII Transactions on Internet and Information Systems* (area editor); *IET Cyber-Physical Systems: Theory & Applications*; *IEEE Open Journal of the Computer Society*; and Wiley S&P.

Jiankun Hu (Senior Member, IEEE) is a full professor of cyber security with the School of Engineering and Information Technology, the University of New South Wales at the Australian Defence Force Academy (UNSW@ADFA), Australia. His major research interest is in computer networking and computer security, especially biometric security. He has been awarded ten Australia Research Council Grants. He served as Security Symposium Co-Chair for IEEE GLOBECOM '08 and IEEE ICC '09. He was the Program Co-Chair of the 2008 International Symposium